

CS3002E Compiler Design

Monsoon 2025

Lecture #26

Translation of Boolean Expressions

Department of Computer Science & Engineering
NIT Calicut

Translation of Boolean expressions

Focuses on translation of boolean expressions that appear as conditional expressions in flow-of-control constructs (like *if-else* and *while*).

- such boolean expressions are not evaluated to values¹.
- the value of the expression is implied by a position in the program (denoted by a label attached to a 3-address instruction).

e.g. in a statement of the form *if (B) S1 else S2*, the boolean expression *B* must be *true* if control reaches *S1* and *B* must be *false* if control reaches *S2*.

¹translation of expressions representing boolean values can be done similar to the translation of arithmetic expressions.

Boolean Expression: Grammar

$$B \rightarrow B || B \mid B \& \& B \mid !B \mid (B) \mid E \text{ relop } E \mid \text{true} \mid \text{false}$$

Boolean Expression: Attributes

For a nonterminal B denoting a boolean expression,

- $B.true$ is the label of the three-address instruction to which control goes if B evaluates to *true*.
- $B.false$ is the label of the three-address instruction to which control goes if B evaluates to *false*.
- $B.code$ denotes the three-address code generated for B .
 $B.code$ consists of 3-address jump (conditional/unconditional) instructions to $B.true/B.false$

Note: $B.true$ and $B.false$ are **inherited** attributes.

Translation of Booleans: example

Code generated for *if* ($a < b$) *small* = a *else* *small* = b

```
    if  $a < b$  goto  $L_1$   
    goto  $L_2$   
 $L_1$ :  $small = a$   
    goto  $L_3$   
 $L_2$ :  $small = b$   
 $L_3$ :
```

Let B denote the boolean expression $a < b$.

- the first two 3-address instructions are generated as $B.code$.
- $B.true$ is L_1 .
- $B.false$ is L_2

Boolean expressions: Short-circuit Evaluation Semantics²

In evaluating a boolean expression B of the form $B_1 \text{ op } B_2$, evaluation of B_2 can be skipped depending on the result of evaluation of B_1 . For example in $B_1 \parallel B_2$, if B_1 evaluates to *true*, then B_2 need not be evaluated.

²*Complete Evaluation Semantics* evaluates both the subexpressions. 

Short-circuit Code

In *short-circuit* (or *jumping*) code, the boolean operators are translated to jump instructions. The operators do not appear in the code.

Translation of $B_1 \&\& B_2$

Example: 3-address code generated for
if ($x < y \&\& x < z$) S1 else S2:

```
    if  $x < y$  goto  $L_1$ 
    goto  $L_3$ 
 $L_1$ : if  $x < z$  goto  $L_2$ 
    goto  $L_3$ 
 $L_2$  : S1
    goto  $L_4$ 
 $L_3$  : S2
 $L_4$  :
```

Translation of $B_1 \&\& B_2$

3-address code generated for
if ($x < y \&\& x < z$) S1 else S2:

```
    if  $x < y$  goto  $L_1$ 
    goto  $L_3$ 
 $L_1$ : if  $x < z$  goto  $L_2$ 
    goto  $L_3$ 
 $L_2$  : S1
    goto  $L_4$ 
 $L_3$  : S2
 $L_4$  :
```

Exercise: Identify the set of instructions generated for each subexpression of the given Boolean Expression.

Translation of $B_1 \&\& B_2$

Production	Semantic Rules
$B \rightarrow B_1 \&\& B_2$	$B_1.true = newLabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel label(B_1.true) \parallel B_2.code$

- If B_1 evaluates to *true*, B_2 should be evaluated. $B_1.true$ is set as the address of the first instruction in $B_2.code$.
- If B_1 evaluates to *false*, the entire expression is *false*, and hence control should go to $B.false$.
- If B_2 evaluates to *true*, control should go to $B.true$.

Translation of other Boolean Expressions

Production	Semantic Rules
$B \rightarrow B_1 \parallel B_2$	$B_1.true = B.true$ $B_1.false = newLabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel label(B_1.false) \parallel B_2.code$
$B \rightarrow !B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$

Exercise

1. For a Boolean expression B , the attributes $B.true$ and $B.false$ are inherited. Draw a parse tree for the following statement:

if ($x < y \ \&\& \ x < z$) S1 else S2

Explain how the inherited attribute values are set for each of the sub expressions of the boolean expression $x < y \ \&\& \ x < z$

2. With the help of the annotated parse tree, explain the code generation for the following statement:

if ($!(x < y)$) S1 else S2

References

References:

- Aho A.V., Lam M.S., Sethi R., and Ullman J.D. Compilers: Principles, Techniques, and Tools (ALSU). Pearson Education, 2007.

Further reading:

- Relevant portions of ALSU Section 6.6